

VOLUME 12 / RUST BACKEND MASTERY

Fly.io 公開 — Docker・CI・運用・復旧

本番用の実行ファイルを作るところから、公開・見守り・元に戻すところまでを、手を動かして確かめる

第12巻: Fly.io 公開 — Docker・CI・運用・復旧

この巻で身に付けること

第11巻までで、RustNote は自分の PC の中で、正しく動くようになりました。この巻では、それを**インターネットに公開し、公開したあとも安全に動かし続ける**方法を学びます。

自分の PC で動くことと、公開して動き続けることの間には、思っているより大きな差があります。公開してから、次のような事故が起きることがよくあります。

- 自分の PC では速かったのに、公開したらログインに何秒もかかる（試し用の作り方のまま公開していた）
- 公開した画面の CSS が読み込まれず、文字だけの画面になる（実行ファイルだけを置いて、見た目のファイルを置き忘れた）
- 「どの版のコードが動いているのか」が、誰にも分からない
- 秘密のトークンを、設定ファイルに書いたまま共有してしまう
- 新しい版を出したら起動しなくなり、前の版に戻す方法が分からない
- DB のデータが消えたのに、元に戻す手順を一度も試していなかった

この巻のテーマは、こうした事故を、**作る手順・置く場所・確かめる方法・戻す方法**を先に決めておくことで防ぐことです。

到達目標

この巻を終えると、次のことができるようになります。

- 本番用の実行ファイル（release ビルド）と、試し用（debug ビルド）の違いを、速さ・大きさ・動きの違いで説明できる。どのコミットと、どの Rust から作った実行ファイルかを記録できる
- multi-stage の Dockerfile で、作る場所と動かす場所を分けた小さなコンテナイメージを作れる
- Fly.io の設定（`fly.toml`）と、アプリが待ち受けるアドレス・ポートをそろえ、秘密の値を `fly secrets` で渡せる
- ヘルスチェックと起動の順番、migration を出す順番を決め、公開したあとにログと動作確認で「本当に動いているか」を確かめられる
- 台数・地域・費用の決め方と、前の版に戻す・DB を元に戻すといった障害のときの手順を、あらかじめ試しておける

学習地図

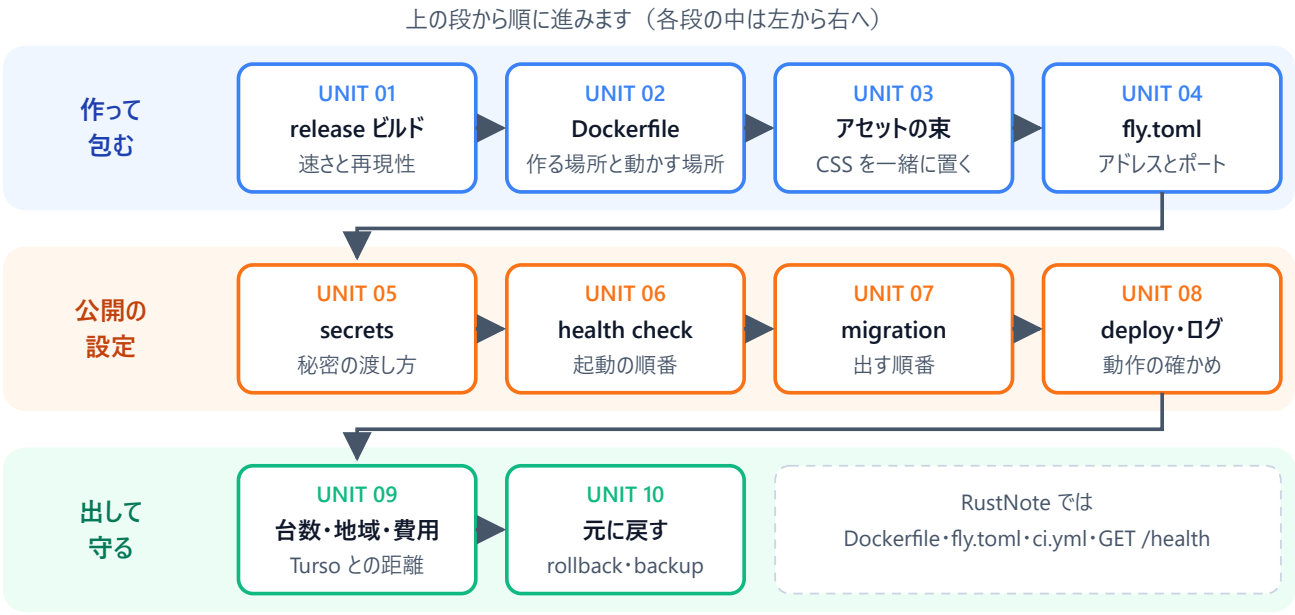


図12-1: 第12巻の学習地図。本番用の実行ファイルを作って包み、公開の設定を決め、出したあとに見守って、困ったら元に戻す

単元	テーマ	ひとことで言うと
UNIT 01	release ビルドと再現性	本番用の実行ファイルを作り、同じソースから同じ物ができるかを確かめる
UNIT 02	multi-stage の Dockerfile（準備中）	作る場所と動かす場所を分け、小さく安全なイメージにする
UNIT 03	アセットの束を本番用に作る（準備中）	`topcoat asset bundle --release` と、`assets/` の置き場所
UNIT 04	fly launch・fly.toml・ポート（準備中）	`0.0.0.0:8080` と `internal_port` をそろえる
UNIT 05	secrets・設定・入れ替え（準備中）	トークンを、イメージにも `fly.toml` にも書かない
UNIT 06	health check と起動の順番（準備中）	「動いている」と「受け付けてよい」を外から確かめる
UNIT 07	migration を出す順番と release command（準備中）	新しい表と古いコードがぶつからないようにする
UNIT 08	deploy・ログ・動作の確かめ・独自ドメイン（準備中）	出したあとに、本当に動いているかを確かめる
UNIT 09	台数・地域・費用・Turso との距離（準備中）	速さと費用の釣り合いを、数で決める
UNIT 10	rollback・backup・障害からの復旧・更新（準備中）	前の版と前のデータに戻す手順を、先に試しておく

> **お知らせ:** この版には UNIT 01 を収録しています。UNIT 02〜10 は準備中で、次の更新から順に追加します。UNIT 04 以降は Fly.io と Turso のアカウントを使って実際に公開して確かめ、その時点の公式の手順に合わせて載せます。

前提知識（先に読んでおく巻）

- **第1巻（環境構築）**：PowerShell・Git・Rust が使えること。この巻の後半では、Docker・flyctl（Fly.io のコマンド）・Turso CLI も使います（それぞれの単元で、入れ方から説明します）。
- **第5巻（async）**：`spawn_blocking` で重い計算を別のスレッドに任せること（第5巻 UNIT 07）。UNIT 01 で、パスワードのハッシュの速さを測ります。
- **第9巻（RustNote 設計）**：RustNote のフォルダ構成と、設定・起動の順番。
- **第13巻（完全コード）UNIT 01:** `Cargo.toml`・`Cargo.lock`・`rust-toolchain.toml` と `--locked`。この巻の UNIT 01 は、その続きとして読めます。

> **注意:** 第10巻・第11巻は、書き直しの途中です。この巻の本文は、教材のフォルダの `Rust/rustnote/` のコードと、この巻で実際に動かした結果だけをもとに書いているので、ほかの巻が未完成でも読み進められます。

この巻の進め方

公開の作業は、一度うまくいくと、手順を忘れてしまいがちです。各単元では、次の順番で進めてください。

1. **予想する:** コマンドを実行する前に、何ができあがり、何が表示されるかを予想します。
2. **そのまま動かす:** 教材のコマンドをそのまま実行し、結果を比べます。
3. **記録する:** 何を、どの版で、いつ作ったかをメモします（UNIT 01 の問3で、記録の作り方を練習します）。
4. **元に戻す:** 試しに変えた設定は、元に戻してから次へ進みます。公開した物を元に戻す方法は、UNIT 10 で学びます。

> **ヒント:** この巻の実行結果には、かかった時間（`1m 25s` や `20.25ms` など）や、ファイルの指紋（SHA-256 の長い英数字）が出てきます。時間は PC の速さで、指紋は作った PC と時刻で変わります。何がなぜ変わるのかは、UNIT 01 で確かめます。

使うツールと版

この巻のコードは、すべて次の環境で実際にビルド・実行し、実行結果とエラーメッセージは実際の出力をそのまま載せています（パソコンのユーザー名などを含む部分は `...` に置き換えています）。

項目	版・内容
OS	Windows 11（ARM64）。コマンドは PowerShell を主に、WSL / macOS / Linux は補足で示します
Rust	**1.98 以上**（本書シリーズの基準）。動作確認は rustc 1.99.0 / cargo 1.99.0（`aarch64-pc-windows-msvc`）
RustNote	教材の `Rust/rustnote/`（`rustnote` 0.1.0）。この巻では `src/` を変えずに使う
topcoat-cli	0.10.0（`topcoat asset bundle` で CSS の束を作る。RustNote の topcoat と同じ版）
Docker	29.3.1（Docker Desktop。UNIT 02 から使います）
flyctl	v0.4.57（Fly.io のコマンド。この PC では `windows/amd64` 版が動きます。UNIT 04 から使います）

練習用のフォルダを用意する

第9巻・第13巻と同じように、教材の `Rust/rustnote/` を**作業用のフォルダにコピー**して使います。この巻では、確かめ用の小さなプログラムを `examples/` に足していきます。RustNote の `src/` は変えません。

```
# 教材の rustnote フォルダを、中身ごと（-Recurse）作業用のフォルダへコピーする
Copy-Item -Recurse "教材のフォルダ\Rust\rustnote" .\rustnote
cd rustnote
# このフォルダで使われる Rust の版を確かめる（1.99.0 になる）
rustup show active-toolchain
```

topcoat CLI がまだ入っていなければ、ここで入れておきます。RustNote の `Dockerfile` と同じ版・同じ書き方です（ダウンロードとコンパイルをするので、数分かかることがあります）。

```
cargo install topcoat-cli --version 0.10.0 --locked
topcoat --version
```

```
topcoat 0.10.0
```

UNIT 01: release ビルドと再現性

この単元のゴール

- debug ビルド（試し用）と release ビルド（本番用）の違いを、速さ・大きさ・整数があふれたときの動きの3つで、実際の数を使って説明できる
- 本番用の実行ファイルを作る手順（`cargo build --release --locked` → `topcoat asset bundle --release`）を説明し、手元で起動して `GET /health` を確かめられる
- 同じソースから2回作った実行ファイルが同じ中身になるか（再現性）を確かめ、違いの原因（リンクした時刻・フォルダの場所）を説明できる
- どのコミット・どの Rust・どの `Cargo.lock` から作った実行ファイルかを、記録として残せる

たとえ話でつかむ

パン屋さんが、新しいパンを売り出すまでを考えます。

- **試作**では、生地をこねるたびに手を止めて、温度や重さを確かめます。焼き上がるまで時間がかかりますが、何かおかしければ、すぐにその場で気づけます。
- **本番の仕込み**では、確かめの手順を減らして、速くたくさん焼きます。そのかわり、試作のときなら止まって気づけた小さな間違いが、そのまま店に並ぶことがあります。
- 焼いたパンだけでは、売れません。**値札と袋**（見た目）を、パンと一緒に棚に並べる必要があります。
- 袋には、**製造記録**（いつ・どのレシピで・どの小麦粉の袋から作ったか）を残します。お客さんから「昨日のパンがおかしい」と言われたとき、どの材料で作ったかをたどれるようにするためです。
- 同じレシピ・同じ材料で焼けば、同じパンになります。ただし、袋に押す**日付の印**だけは、毎回違います。

RustNote の公開の準備も、同じ形をしています。

パン屋さん	RustNote
試作（確かめながら、ゆっくり）	**debug ビルド**（`cargo build`。あふれの確かめ付き・最適化なし）
本番の仕込み（速く、たくさん）	**release ビルド**（`cargo build --release`。最適化あり・あふれの確かめなし）
値札と袋	**アセットの束**（`topcoat asset bundle --release` が作る `assets/`）
製造記録	**ビルドの記録**（コミット・Rust の版・`Cargo.lock` と実行ファイルの指紋）
袋の日付の印	実行ファイルに入る**リンクした時刻**

> **用語:** debug ビルド（デバッグビルド： `cargo build` で作る、試し用の実行ファイル。コンパイルが速く、間違いに気づきやすいが、動きは遅い。`target\debug` にできる）

> **用語:** release ビルド（リリースビルド： `cargo build --release` で作る、本番用の実行ファイル。コンパイルに時間をかけて最適化し、速く動く。`target\release` にできる）

> **用語:** 再現性（さいげんせい：同じソース・同じ道具・同じ部品から作れば、いつ誰が作っても同じ物ができること。英語では reproducibility）

図で理解する

同じ RustNote のソースから作った2つの実行ファイルを、この PC で比べると、次のようになりました（数は、この単元の例で実際に測ったものです）。

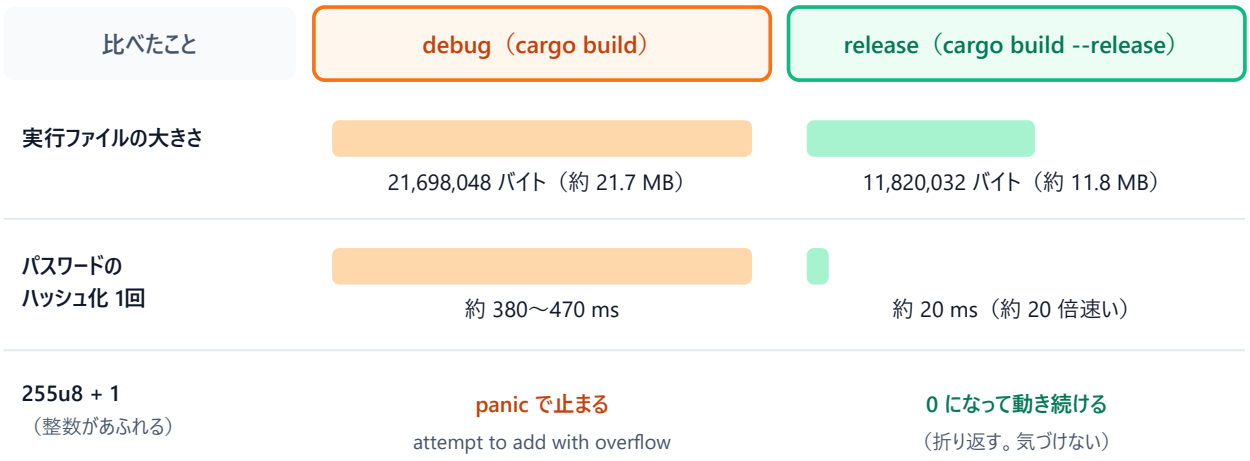


図12-2: 同じソースから作った debug と release の違い。release は小さく速いが、整数のあふれを確かめない。どの数も、この PC（Windows ARM64）で実際に測ったもの

- **大きさ:** release は、debug の約半分でした。最適化で、使われない部分や、確かめのためのコードが減るからです。
- **速さ:** パスワードのハッシュ化（Argon2id）は、release のほうが約 20 倍速くなりました。ログインのたびに行う計算なので、この差は、利用者が待つ時間にそのまま出ます。
- **あふれ:** debug では、整数があふれると panic で止まります。release では、何も言わずに折り返して（`255 + 1` が `0` になって）動き続けます。release のほうが、間違いに気づきにくいのです（エラー3）。

本番用の実行ファイルは、次の順番で作ります。RustNote の `Dockerfile` も、同じ順番で作っています（UNIT 02（準備中））。

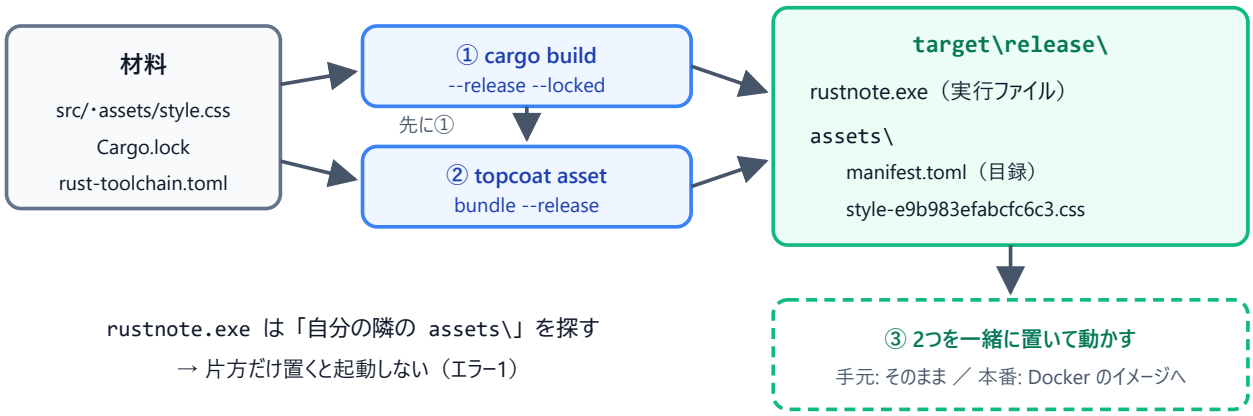


図12-3: 本番用の実行ファイルができるまで。① release ビルドの後に ② アセットの束を作り、2つを必ず一緒に置く

- ① **release ビルド**で、`target\release\rustnote.exe` ができます。`--locked` で、`Cargo.lock` のとおりの版を使います（第13巻 UNIT 01）。
- ② **アセットの束**は、`topcoat asset bundle --release` で作ります。topcoat CLI は、①でできた実行ファイルの中から `asset!("assets/style.css")` の印を探し、CSS を `style-e9b983efabcfc6c3.css` のような名前（中身から計算した指紋付き）にして、実行ファイルの隣の `assets\` に置きます。だから、①が先です（くわしくは UNIT 03（準備中））。
- ③ 実行ファイルは、起動するとき「**自分の隣の** `assets\` 」を読みます。片方だけをコピーすると、起動しません。

まず動かす最小コード

例1-1: release ビルドを作り、起動して確かめる

> **▼ このコマンドがやること（先に日本語で）:** `Cargo.lock` のとおりの版で release ビルドを作り、CSS の束を作ってから、できた実行ファイルを起動します。別の PowerShell の窓から `GET /health` を送り、`{"status":"ok"}` が返ることを確かめます。

```
# ① 本番用にビルドする（Cargo.lock を書き換えない）
cargo build --release --locked
# ② 実行ファイルの隣に、CSS の束を作る
topcoat asset bundle --release
# ③ 起動する（止めるときは Ctrl+C）
.\target\release\rustnote.exe
```

▼ 実行結果（`cargo build` は `Compiling` の行が 289 行続くので、最後の2行だけを載せています。クレートをまだダウンロードしていない PC では、その表示も加わります）

```
   Compiling rustnote v0.1.0 (...\rustnote)
    Finished `release` profile [optimized] target(s) in 1m 25s
```

```
bundled assets into target\release\assets
```

```
マイグレーションを適用しました: 0001_users
マイグレーションを適用しました: 0002_notes
マイグレーションを適用しました: 0003_sessions
RustNote を起動しました: http://127.0.0.1:3000
```

起動したまま、もう1つの PowerShell の窓で、ヘルスチェックの URL を開きます。`curl.exe` の `-i` は、本文の前に、返事の見出し（ヘッダー）も表示する指定です。

```
curl.exe -i http://127.0.0.1:3000/health
```

```
HTTP/1.1 200 OK
content-type: application/json
content-length: 15
date: Mon, 26 Oct 2026 20:41:33 GMT

{"status":"ok"}
```

▼ コードを1つずつ分解して解説

解説1: release は target¥release にできる

- `Finished release profile [optimized]` の `optimized` は「最適化した」という意味です。debug ビルドのときは `Finished dev profile [unoptimized + debuginfo]`（最適化なし・デバッグ用の情報付き）と表示されます。

- debug ビルドは `target\debug\`、release ビルドは `target\release\` と、別のフォルダにできます。2つは混ざりません。この PC では、何もない状態からの release ビルドに 1 分 25 秒かかりました。debug ビルドは 1 分 2 秒でした。release は、最適化に時間を使うぶん遅くなります。

解説2: bundle は「実行ファイルの隣」に作る

- `bundled assets into target\release\assets` は、「`target\release\assets` に束を作りました」です。`--release` を付けたので、release の実行ファイルの隣にできました。付け忘れると `target\debug\assets` にできてしまいます（エラー1）。
- 実際の表示では、`target` の前に、フォルダの場所が付きます。

解説3: 起動の表示とヘルスチェック

- 新しい DB ファイル（`rustnote.db`）だったので、起動のときに3つのマイグレーションが適用されました。2回目からは、この3行は出ません。DB ファイルは、コマンドを実行したフォルダ（今いるフォルダ）にできます。`.env` が無いときの `DATABASE_URL` の既定値が `rustnote.db` だからです（第13巻 UNIT 01）。
- `GET /health` は 200 で `{"status":"ok"}` を返しました。RustNote の `/health` は、DB に `SELECT 1` を送って、答えが返れば `ok` です。Fly.io も、公開したあと、この URL で「動いているか」を確かめます（UNIT 06（準備中））。

例1-2: debug と release の速さを比べる

> **▼ このコードがやること（先に日本語で）**: RustNote の `auth::hash_password`（パスワードを Argon2id でハッシュ化する）と `auth::verify_password`（照合する）を3回ずつ呼び、それぞれにかかった時間を表示します。同じプログラムを、debug と release の2通りで動かして比べます。

```rust
//! RustNote のパスワードのハッシュ化と照合（Argon2id）にかかる時間を、3回ずつ測る。
//! debug と release で比べる:
//!   cargo run -q --example u01_hash_speed
//!   cargo run -q --release --example u01_hash_speed

use std::time::Instant;

use rustnote::auth::{hash_password, verify_password};
use rustnote::domain::Password;

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    // debug ビルドでは debug_assertions が有効、release ビルドでは無効になる
    let profile = if cfg!(debug_assertions) {
        "debug"
    } else {
        "release"
    };
    println!("ビルドの種類: {profile}");

    let password = Password::parse("correct horse battery")?;
    for n in 1..=3 {
        let started = Instant::now();
        let phc = hash_password(&password).await?;
        let hashed = started.elapsed();

        let started = Instant::now();
        let matches = verify_password("correct horse battery", &phc).await?;
        let verified = started.elapsed();

        println!("{n}回目: ハッシュ化 {hashed:.2?}・照合 {verified:.2?}（一致:
{matches}）");
    }
    Ok(())
}
```

`examples/u01_hash_speed.rs` として保存し、`--release` を付けずに1回、付けて1回実行します。

```
cargo run -q --example u01_hash_speed
cargo run -q --release --example u01_hash_speed
```

▼ 実行結果

```
ビルドの種類: debug
1回目: ハッシュ化 473.06ms・照合 387.84ms（一致: true）
2回目: ハッシュ化 392.99ms・照合 401.03ms（一致: true）
3回目: ハッシュ化 380.45ms・照合 399.83ms（一致: true）
```

```
ビルドの種類: release
1回目: ハッシュ化 21.19ms・照合 19.33ms（一致: true）
2回目: ハッシュ化 20.25ms・照合 19.28ms（一致: true）
3回目: ハッシュ化 20.03ms・照合 19.63ms（一致: true）
```

▼ コードを1つずつ分解して解説

解説1: cfg!(debug_assertions) で、どちらのビルドかが分かる

- `cfg!(debug_assertions)` は、debug ビルドでは `true`、release ビルドでは `false` になります。`debug_assert!`（debug のときだけ確かめる `assert!`）は、この設定で有効・無効が切り替わります。整数のあふれの確かめも、既定では debug で有効・release で無効です（エラー3）。

解説2: 約 20 倍の差は、ログインの待ち時間の差

- ハッシュ化は、debug で約 380〜470 ms、release で約 20 ms でした。Argon2id は、わざと計算を重くして、パスワードを総当たりで試されにくくする仕組みです（第8巻）。重い計算なので、最適化の効き目がとても大きく出ます。
- debug のままの実行ファイルを公開すると、ログインと登録のたびに、利用者を 0.4 秒近く余計に待たせることになります。しかも、そのあいだ CPU を使い続けるので、同時にログインする人が増えると、もっと遅くなります。**公開するのは release ビルド**です。

解説3: 1回目だけ少し遅い

- debug の1回目のハッシュ化（473.06 ms）は、2回目以降より少し遅くなりました。最初の1回は、計算に使うメモリを用意したり、`spawn_blocking` 用のスレッドを作ったりする分が加わるからだと考えられます。速さを比べるときは、1回だけで決めず、何回か測ります。

もう一歩: 実務に近い形

公開した実行ファイルについて、「これは、どのソースから作った物か」を後から確かめられることが大切です。同じソースから作れば、いつでも同じ実行ファイルになるのでしょうか。確かめるための小さな道具を作ります。

例1-3: 実行ファイルを調べる道具を作る

> **▼ このコードがやること（先に日本語で）**：渡された実行ファイル（.exe）を読み、大きさ、リンクした時刻（実行ファイルの見出しに書かれている）、ユーザーフォルダの場所（`C:\Users\あなたの名前` の部分）が中に何か所入っているかを表示します。2つ渡すと、中身が何バイト違うかも数えます。外のクレートは使わず、標準ライブラリだけで書きます。

```rust
//! Windows の実行ファイル（.exe）を調べて、大きさ・リンクした時刻・ユーザーフォルダの
//! 場所が何か所入っているかを表示する。2つ渡すと、違うバイトの数も数える。
//! 実行: cargo run -q --example u01_exe_report -- 1つ目.exe [2つ目.exe]

use std::env;
use std::fs;

/// PE 形式（Windows の実行ファイルの形式）の見出しから、リンクした時刻を読む。
/// 時刻は「1970-01-01 からの秒数」（UNIX 時刻）で入っている
fn link_time(bytes: &[u8]) -> Option<u32> {
    // 先頭から 0x3C の位置に、PE の見出しがどこから始まるかが書いてある
    let header: [u8; 4] = bytes.get(0x3C..0x40)?.try_into().ok()?;
    let pe = usize::try_from(u32::from_le_bytes(header)).ok()?;
    // 見出しは "PE\0\0" で始まり、その 8 バイト後ろに時刻がある
    if bytes.get(pe..pe + 4)? != b"PE\0\0" {
        return None;
    }
    let time: [u8; 4] = bytes.get(pe + 8..pe + 12)?.try_into().ok()?;
    Some(u32::from_le_bytes(time))
}

/// `needle` が何か所に入っているかを数える
fn count(haystack: &[u8], needle: &[u8]) -> usize {
    haystack
        .windows(needle.len())
        .filter(|window| *window == needle)
        .count()
}

fn main() -> std::io::Result<()> {
    let paths: Vec<String> = env::args().skip(1).collect();
    // ユーザーフォルダの場所（例: C:\Users\taro）。場所そのものは表示しない
    let profile = env::var("USERPROFILE").unwrap_or_default();

    let mut files = Vec::new();
    for path in &paths {
        let bytes = fs::read(path)?;
        println!("[{path}]");
        println!("    大きさ: {} バイト", bytes.len());
        match link_time(&bytes) {
            Some(time) => println!("    リンクした時刻（UNIX 秒）: {time}"),
            None => println!("    リンクした時刻: 読めません（PE 形式ではない）"),
        }
        if !profile.is_empty() {
            let found = count(&bytes, profile.as_bytes());
            println!("    ユーザーフォルダの場所が入っている数: {found} か所");
        }
        files.push(bytes);
    }
```

```
    if let [first, second] = files.as_slice() {
        let same_length = first.len() == second.len();
        let differ = first.iter().zip(second).filter(|(a, b)| a != b).count();
        println!("--- 2つを比べる");
        println!("    大きさが同じ: {same_length}");
        println!("    違うバイトの数: {differ}");
    }
    Ok(())
}
```

同じソースから、release ビルドを2回作って比べます。2回目は、RustNote の部分だけを消してから作り直します（依存クレートは作り直さないので、1回目より速く終わります）。

```
# 1回目の実行ファイルを、builds フォルダに取っておく
New-Item -ItemType Directory builds
Copy-Item target\release\rustnote.exe builds\build1.exe
# RustNote の部分だけを消して、作り直す
cargo clean --release -p rustnote
cargo build --release --locked
Copy-Item target\release\rustnote.exe builds\build2.exe
# 2つを比べる
cargo run -q --example u01_exe_report -- builds/build1.exe builds/build2.exe
```

▼ 実行結果

```
[builds/build1.exe]
    大きさ: 11820032 バイト
    リンクした時刻（UNIX 秒）: 1793046265
    ユーザーフォルダの場所が入っている数: 1820 か所
[builds/build2.exe]
    大きさ: 11820032 バイト
    リンクした時刻（UNIX 秒）: 1793046676
    ユーザーフォルダの場所が入っている数: 1820 か所
--- 2つを比べる
    大きさが同じ: true
    違うバイトの数: 24
```

▼ コードを1つずつ分解して解説

解説1: 同じソースなのに、24 バイト違った

- 2つの実行ファイルは、大きさは同じですが、中身が 24 バイト違いました。SHA-256 の指紋（`Get-FileHash` で表示できる）も、まったく別の値になります。

- 違いは、**リンクした時刻**でした。1793046265 と 1793046676 の差は 411 秒で、この PC で2回のビルドの間にたった時間です。Windows の実行ファイル（PE 形式）の見出しには、作った時刻が入ります。違う 24 バイトの場所を1つずつ調べると、見出しの時刻（2 バイト）、デバッグ情報の目次に入っている同じ時刻（2 バイトずつ3か所）、デバッグ情報のファイル（`rustnote.pdb`）と組み合わせるための識別番号（16 バイト）でした。プログラムの命令の部分は、1 バイトも違いませんでした。
- つまり、**コードは同じでも、作るたびに指紋が変わる**ということです。これでは、「公開した実行ファイルが、本当にこのソースから作った物か」を、もう一度作って指紋を比べる方法で確かめられません。

解説2: /Brepro を付けると、同じ中身になる

- Windows のリンカー（`link.exe`）には、時刻の代わりに中身から計算した値を使う `/Brepro` という指定があります。Rust から渡すには、環境変数 `RUSTFLAGS` を使います。`RUSTFLAGS` を変えると、全部のクレートが作り直しになるので、別の `--target-dir` で試します。

```
$env:RUSTFLAGS = "-C link-arg=/Brepro"
cargo build --release --locked --target-dir target\brepro
Copy-Item target\brepro\release\rustnote.exe builds\brepro1.exe
cargo clean --release -p rustnote --target-dir target\brepro
cargo build --release --locked --target-dir target\brepro
Copy-Item target\brepro\release\rustnote.exe builds\brepro2.exe
# 試し終わったら、環境変数を消す
Remove-Item Env:RUSTFLAGS
cargo run -q --example u01_exe_report -- builds/brepro1.exe builds/brepro2.exe
```

```
[builds/brepro1.exe]
    大きさ: 11820032 バイト
    リンクした時刻（UNIX 秒）: 277490482
    ユーザーフォルダの場所が入っている数: 1820 か所
[builds/brepro2.exe]
    大きさ: 11820032 バイト
    リンクした時刻（UNIX 秒）: 277490482
    ユーザーフォルダの場所が入っている数: 1820 か所
--- 2つを比べる
    大きさが同じ: true
    違うバイトの数: 0
```

- 違うバイトの数が 0 になりました。「リンクした時刻」は、本当の時刻ではなく、中身から計算した値（277490482）になっています。これで、同じソースから作れば同じ指紋になりました。

解説3: 実行ファイルに、自分のフォルダの場所が残っている

- 「ユーザーフォルダの場所が入っている数」は、この PC では 1,820 か所でした。panic のメッセージに使う、ソースファイルの場所（`C:\Users\あなたの名前\.cargo\registry\...` など）が、実行ファイルの中に文字として残っているのです。この実行ファイルを人に渡すと、あなたのユーザー名も一緒に渡ることになります。
- `--remap-path-prefix` を使うと、実行ファイルに入れる場所の文字を、別の文字に置き換えられます。

```
$env:RUSTFLAGS = "-C link-arg=/Brepro --remap-path-prefix=$env:USERPROFILE=~"
cargo build --release --locked --target-dir target\remap
Copy-Item target\remap\release\rustnote.exe builds\remap.exe
Remove-Item Env:RUSTFLAGS
cargo run -q --example u01_exe_report -- builds/remap.exe
```

```
[builds/remap.exe]
    大きさ: 11795456 バイト
    リンクした時刻（UNIX 秒）: 2004121190
    ユーザーフォルダの場所が入っている数: 1 か所
```

- 1,820 か所が 1 か所に減りました。残った1か所は、RustNote のフォルダの場所です。Topcoat の `asset!` マクロは、`assets/style.css` のような場所を、あとで topcoat CLI が見つけられるように、**クレートのフォルダの場所**（`CARGO_MANIFEST_DIR`）を実行ファイルに書き込みます（topcoat-asset 0.10.0 の `asset!` のソースで確かめました）。これは、アセットの束を作るために必要な情報です。
- この単元の実験で分かったのは、**作る場所（フォルダの場所・ユーザー名・時刻）が、実行ファイルの中身に入り込む**ということです。そこで、本番用の実行ファイルは、自分の PC ではなく、**いつも同じ場所・同じ道具で作る**ようにします。RustNote の `Dockerfile` が、`rust:1.99.0-bookworm` のイメージの中の `/app` フォルダでビルドしているのは、このためです（UNIT 02（準備中））。



図12-4: 作る場所が実行ファイルに入り込む様子。リンカーとコンパイラーの指定で減らせるが、本番用はいつも同じ場所（Docker の中）で作るのが確実

よくあるエラーと読み方

エラー1: アセットの束がない（no asset bundle at ...）

release ビルドを作った直後に、`topcoat asset bundle --release` をせずに起動した場合です。`--release` を付けずに `topcoat asset bundle` だけを実行した場合も、同じエラーになります（束が `target\debug\assets` にできて、release の実行ファイルの隣には無いからです）。

```
topcoat asset bundle
.\target\release\rustnote.exe
```

```
bundled assets into target\debug\assets
```

```
Error: アセットの束（assets/）がありません。先に `topcoat asset bundle` を実行するか、`topcoat dev` で起動してください

Caused by:
    no asset bundle at target\release\assets
```

- **何が起きたか:** `Error:` の行は、RustNote の `main.rs` が `anyhow` の `context` で付けた説明です。`Caused by:` の下の `no asset bundle at target\release\assets` が、Topcoat の `AssetBundle::load()` が返した、もとの原因です。実行ファイルの隣（`target\release\assets`）に束がありませんでした。
- **DB には触っていない:** RustNote の `main.rs` は、アセットを読み込んでから DB を開きます。だから、このエラーのときは、`rustnote.db` も作られていません。準備に失敗するものを、先に確かめているのです（第9巻 UNIT 06（準備中））。
- **直し方:** `topcoat asset bundle --release` を実行してから、もう一度起動します。
- **RustNote の改善点:** エラーの文は「先に `topcoat asset bundle` を実行するか」と案内していますが、release の実行ファイルのときは `--release` も必要です。「release のときは `--release` を付ける」まで書いておくと、迷う人が減ります（RustNote のコードは変えていません）。

エラー2: ポートがもう使われている（os error 10048）

RustNote が動いているのに、別の窓から、もう1つ起動した場合です。

```
.\target\release\rustnote.exe
```

```
RustNote を起動しました: http://127.0.0.1:3000
Error: サーバーを起動できませんでした（HOST / PORT を確認してください）

Caused by:
    通常、各ソケット アドレスに対してプロトコル、ネットワーク アドレス、またはポートのどれか 1 つのみを使用できます。 (os error 10048)
```

- **何が起きたか:** `通常、各ソケット アドレスに対して…1 つのみを使用できます。(os error 10048)` は、Windows の「そのアドレスとポート（127.0.0.1:3000）は、もうほかのプログラムが使っています」というエラーです。Linux では `Address already in use (os error 98)` のような文になります。

- **直し方**: 先に動いている RustNote を Ctrl+C で止めるか、`$env:PORT = "3001"` のように、別のポートを指定してから起動します（`topcoat::start` は `HOST` と `PORT` の環境変数を読みます）。
- **気をつけたい点**: 1行目に `RustNote を起動しました: http://127.0.0.1:3000` と出ています。ところが、このプログラムは起動できずに止まりました。RustNote の `main.rs` は、ポートを押さえる（`topcoat::start`）**前に**「起動しました」を表示しているからです。ログを見る人が「起動した」と勘違いしないよう、ポートを押さえたあとに表示するのが本来の形です。これも、RustNote に残っている改善点です（コードは変えていません）。

エラー3（エラーが出ない違い）: 整数のあふれが、release では折り返す

`u8`（0〜255 の整数）の 255 に 1 を足す、小さなプログラムです（`examples/u01_overflow.rs`）。

```
//! 整数のあふれ（オーバーフロー）が、debug と release でどう違うかを確かめる。
//!   cargo run -q --example u01_overflow
//!   cargo run -q --release --example u01_overflow

use std::hint::black_box;

fn main() {
    // black_box は、コンパイラに値を先に計算させないための箱（実行してから値が決まる、という扱いにする）
    let count: u8 = black_box(255);
    println!("count = {count}");
    let next = count + 1;
    println!("count + 1 = {next}");
}
```

```
cargo run -q --example u01_overflow
cargo run -q --release --example u01_overflow
```

```
count = 255

thread 'main' (125244) panicked at examples\u01_overflow.rs:11:16:
attempt to add with overflow
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

```
count = 255
count + 1 = 0
```

- **何が起きたか**: debug では `attempt to add with overflow`（足し算であふれようとしました）の panic で止まりました。release では、何も言わずに `0` になりました。`u8` の 256 は表せないので、0 に折り返したのです。
- `black_box` を使っているのは、`255 + 1` のように、コンパイルのときに答えが分かる式だと、コンパイラーが先にエラーにしてしまうからです。実際のプログラムでは、値はフォームや DB から来るので、コンパイラーは気づけません。

- **これが怖い理由:** テスト（debug）では panic で気づけたのに、本番（release）では、まちがった数のまま動き続けます。お金や件数の計算で起きると、静かにデータが壊れます。
- **直し方は2通り:** 1つは、あふれる可能性のある計算を、`checked_add`（あふれたら `None`）や、大きい型（`u64`）で計算することです（問2）。もう1つは、`Cargo.toml` に次の設定を書いて、release でもあふれを確かめるようにすることです。少し遅くなるかわりに、debug と同じく panic で止まります。

```
[profile.release]
overflow-checks = true
```

この設定は、`Cargo.toml` を書き換えずに、環境変数 `CARGO_PROFILE_RELEASE_OVERFLOW_CHECKS=true` で試すこともできます。試すと、release でも debug と同じ panic になりました。

```
count = 255

thread 'main' (130896) panicked at examples\u01_overflow.rs:11:16:
attempt to add with overflow
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

RustNote ではここで使う

RustNote の `Dockerfile` の「1. builder」の部分は、この単元の手順を、そのまま並べた形です（抜き出しています）。

```
FROM rust:1.99.0-bookworm AS builder
WORKDIR /app

# アセットの束を作る topcoat CLI（アプリの Topcoat と同じ 0.10.0）
RUN cargo install topcoat-cli --version 0.10.0 --locked

# ソースをコピーしてビルドする（Cargo.lock の版をそのまま使う）
COPY . .
RUN cargo build --release --locked

# 実行ファイルの隣（target/release/assets/）に CSS の束を作る
RUN topcoat asset bundle --release
```

- Rust の版（1.99.0）と、作るフォルダ（`/app`）が、いつも同じです。例1-3で見た「作る場所が中身に入り込む」問題が、毎回同じ形になります。
- `cargo build --release --locked` → `topcoat asset bundle --release` の順番も、例1-1と同じです。

整数のあふれについても、RustNote は気をつけて書いてあります。`src/domain.rs` の `NoteQuery` です（抜き出しています）。

```
            page: page.unwrap_or(1).max(1),
```

```
    pub fn offset(&self) -> u64 {
        u64::from(self.page - 1) * u64::from(PER_PAGE)
    }
```

- `page.unwrap_or(1).max(1)` で、ページ番号を必ず 1 以上にしています。だから `self.page - 1` が 0 より小さくなる（`u32` で引き算があふれる）ことはありません。`?page=0` が来ても、1 ページ目として扱います。
- 何件飛ばすか（`offset`）は、`u32` のままではなく `u64` に広げてから掛けています。`u32` の最大値どうしを掛けても、`u64` ならあふれません。なぜこう書くのかは、問2で確かめます。

練習問題

問1: release ビルドの `rustnote.exe` だけを、別のフォルダ（たとえば `dist`）にコピーして起動すると、どうなるでしょうか。予想してから確かめてください。そのあと、そのフォルダで RustNote を起動できるようにするには、何をコピーすればよいでしょうか。起動できたら、DB ファイルはどこにできたかも確かめてください。

問2: もし `NoteQuery::offset` を `u64` に広げずに、`u32` のまま `(self.page - 1) * PER_PAGE` と書いていたら、URL に `?page=214748366` を付けて開いたとき、debug と release でそれぞれどうなるでしょうか。小さなプログラムで確かめてください。

問3: 公開した実行ファイルについて、「どのコミット・どの Rust・どの `Cargo.lock` から作ったか」を後から確かめられるように、PowerShell で記録のファイル（`release-record.txt`）を作ってください。練習用のフォルダを Git で管理している（`git init` してコミットしてある）ものとします。

解答と解説

問1の解答:

予想: 起動しない。実行ファイルは「自分の隣の `assets\`」を読むのに、`dist` には `assets\` が無いからです。

```
New-Item -ItemType Directory dist
Copy-Item target\release\rustnote.exe dist\
cd dist
.\rustnote.exe
```

```
Error: アセットの束（assets/）がありません。先に `topcoat asset bundle` を実行するか、`topcoat dev` で起動してください

Caused by:
    no asset bundle at ...\dist\assets
```

- 今いるフォルダではなく、**実行ファイルのある場所**（`dist\assets`）を探しました。`target\release\assets` があっても、使われません。
- `target\release\assets` を、フォルダごと `dist` にコピーすると起動できます。`Copy-Item -Recurse ..\target\release\assets .\assets` のあと、もう一度起動して `/health` を開くと、`{"status":"ok"}` が返りました。

```
マイグレーションを適用しました: 0001_users
マイグレーションを適用しました: 0002_notes
マイグレーションを適用しました: 0003_sessions
RustNote を起動しました: http://127.0.0.1:3000
```

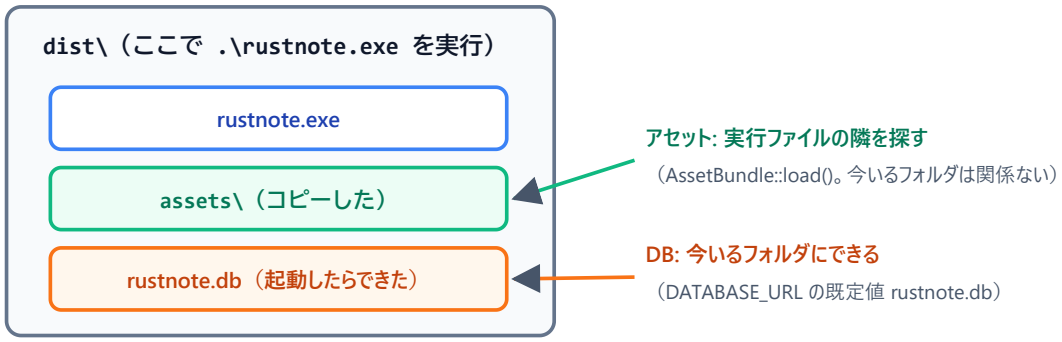


図12-5: 探す場所の決まりの違い。アセットは実行ファイルの隣を、DB ファイルは今いるフォルダを使う

- 起動したあとの `dist` には、`rustnote.exe`・`assets`・`rustnote.db` の3つがありました。DB ファイルは、今いるフォルダ（`dist`）にできます。アセットは「実行ファイルの隣」、DB は「今いるフォルダ」と、探す場所の決まりが違うことに注意してください。本番では、DB の場所を `DATABASE_URL` で決めておきます（RustNote の `Dockerfile` は `/data/rustnote.db`）。

問2の解答:

`examples/u01_q2.rs` です。RustNote と同じ `u64` の計算と、`checked_mul`、`u32` のままのふつうの `*` を並べます。

```rust
//! 問2の解答: ページ番号から「何件飛ばすか」（offset）を計算するとき、
//! u32 のままだとあふれる。RustNote の NoteQuery::offset と比べる。
//!   cargo run -q --example u01_q2
//!   cargo run -q --release --example u01_q2

use std::hint::black_box;

const PER_PAGE: u32 = 20;

fn main() {
    // URL の ?page=214748366 から来たページ番号
    let page: u32 = black_box(214_748_366);

    // RustNote と同じ: u64 に広げてから掛ける（u32 の最大値どうしを掛けてもあふれない）
    let wide = u64::from(page - 1) * u64::from(PER_PAGE);
    println!("u64 に広げて計算: {wide}");

    // checked_mul は、あふれたら None を返す
    println!(
        "u32 のまま checked_mul: {:?}",
        (page - 1).checked_mul(PER_PAGE)
    );

    // u32 のまま、ふつうの * で掛ける
    let narrow = (page - 1) * PER_PAGE;
    println!("u32 のまま *: {narrow}");
}
```

```
cargo run -q --example u01_q2
cargo run -q --release --example u01_q2
```

```
u64 に広げて計算: 4294967300
u32 のまま checked_mul: None

thread 'main' (123884) panicked at examples\u01_q2.rs:25:18:
attempt to multiply with overflow
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

```
u64 に広げて計算: 4294967300
u32 のまま checked_mul: None
u32 のまま *: 4
```

- 正しい答えは 4,294,967,300 件飛ばすこと（`u64` の結果）です。これは `u32` の最大値（4,294,967,295）を少しだけ超えます。

- debug では、掛け算があふれて panic しました。RustNote のページの処理の中なら、500 の返事になります（第9巻 UNIT 04 の寄り道）。
- release では、`4` になりました。4,294,967,300 から 4,294,967,296 を引いた余りです。つまり、**とても大きなページ番号を付けただけで、5 件目からのノートが表示されてしまう**のです。エラーにもならないので、気づくのはとても難しくなります。
- `checked_mul` は、あふれるときに `None` を返すので、「あふれた」ことをプログラムで扱えます。RustNote のように、はじめから大きな型で計算するのも、確実な方法です。

問3の解答:

```
$exe = "target\release\rustnote.exe"
@(
    "commit:     $(git rev-parse --short HEAD)"
    "rustc:      $(rustc --version)"
    "Cargo.lock: $((Get-FileHash Cargo.lock).Hash)"
    "rustnote:   $((Get-FileHash $exe).Hash)"
    "built at:   $((Get-Item $exe).LastWriteTimeUtc.ToString('u'))"
) | Set-Content -Encoding utf8 release-record.txt
Get-Content release-record.txt
```

```
commit:     9df9eca
rustc:      rustc 1.99.0 (b940084d7 2026-09-28)
Cargo.lock: 720833E50B0032AC3F7822FBF12C1D55515AA1D54B90E31C6C32CB99F9541D2D
rustnote:   6B6F18B361A388AF072E5195627E80B99D4FD4DB450278B5C948C5FCE2266823
built at:   2026-10-26 20:31:16Z
```

- `git rev-parse --short HEAD` は、今のコミットの短い名前です。`rustc --version` は、このフォルダで使われる Rust の版（`rust-toolchain.toml` の 1.99.0）です。`Get-FileHash` は、ファイルの SHA-256 の指紋を計算します。
- `built at` は、実行ファイルを最後に書き換えた時刻（UTC）です。この例では、例1-3で調べた「リンクした時刻」（1793046676 秒）と同じ、2026-10-26 20:31:16（UTC）になりました。
- この記録があれば、本番で問題が起きたとき、「このコミットを、この Rust と、この `Cargo.lock` で作った物」とたどれます。`/Brepro` などで再現できるように作ってあれば、同じコミットからもう一度作って、`rustnote` の指紋が一致するかで確かめることもできます。
- 記録のファイルは、Git に入れるより、実行ファイルと一緒に保管するほうが向いています（コミットの名前を、そのコミットの中に書くことはできないからです）。

まとめ

- 公開するのは release ビルド。この PC では、debug より大きさが約半分、パスワードのハッシュ化が約 20 倍速かった。ただし release は整数のあふれを確かめないので、大きい型や `checked_*` で計算するか、`overflow-checks = true` を設定する。

- 本番用の実行ファイルは、`cargo build --release --locked` → `topcoat asset bundle --release` の順で作り、実行ファイルと `assets\` を必ず一緒に置く。実行ファイルは「自分の隣の `assets\` 」を、DB は「今いるフォルダ」（または `DATABASE_URL` ）を使う。
- 作る時刻と場所は、実行ファイルの中身に入り込む。同じソースから同じ物を作れるよう、作る場所と道具を固定し（Docker）、コミット・Rust の版・`Cargo.lock` ・実行ファイルの指紋を記録しておく。

この巻のまとめ

この版に収録した UNIT 01 で、次のことができるようになったかを確かめましょう。できたものに印を付けてください。

UNIT 01: release ビルドと再現性

- ☐ debug と release の違いを、大きさ（約 21.7 MB と 11.8 MB）・速さ（ハッシュ化 約 400 ms と 20 ms）・整数のあふれ（panic と折り返し）で説明できる
- ☐ `cargo build --release --locked` → `topcoat asset bundle --release` の順で本番用の実行ファイルを作り、起動して `GET /health` で確かめられる
- ☐ 実行ファイルは「自分の隣の `assets\` 」を、DB は「今いるフォルダ」（または `DATABASE_URL` ）を使うことを説明できる
- ☐ `no asset bundle at ...` と `os error 10048` のエラーを読み、直し方を言える
- ☐ 同じソースから作った実行ファイルの中身が違う理由（リンクした時刻・識別番号）と、作る場所が中身に入り込むこと（ユーザーフォルダの場所・`asset!` のクレートの場所）を、実際の比べ方で説明できる
- ☐ release でも整数のあふれに気づけるよう、大きい型・`checked_*` ・`overflow-checks = true` のどれかを選べる
- ☐ コミット・Rust の版・`Cargo.lock` と実行ファイルの指紋を、`release-record.txt` に記録できる

> **お知らせ:** この版には UNIT 01 を収録しています。UNIT 02（multi-stage の Dockerfile）・UNIT 03（アセットの束を本番用に作る）・UNIT 04（fly launch・fly.toml・ポート）・UNIT 05（secrets・設定・入れ替え）・UNIT 06（health check と起動の順番）・UNIT 07（migration を出す順番と release command）・UNIT 08（deploy・ログ・動作の確かめ・独自ドメイン）・UNIT 09（台数・地域・費用・Turso との距離）・UNIT 10（rollback・backup・障害からの復旧・更新）は準備中で、次の更新から順に追加します。それまでの間、公開の全体の流れは、RustNote の `README.md` の「公開する（Fly.io + Turso）の流れ」と、`Dockerfile` ・`fly.toml` の先頭のコメントで確かめられます。

用語ミニ辞典

この巻（UNIT 01）に出てきた用語です。

UNIT 01 の用語

用語	意味
debug ビルド	`cargo build` で作る試し用の実行ファイル。最適化なし、整数のあふれを確かめる。`target\debug` にできる
release ビルド	`cargo build --release` で作る本番用の実行ファイル。最適化あり、既定では整数のあふれを確かめない。`target\release` にできる
最適化	同じ動きのまま、速く小さくなるように、コンパイラーがプログラムを作り直すこと
`cfg!(debug_assertions)`	debug ビルドなら `true`、release ビルドなら `false` になる式
整数のあふれ（オーバーフロー）	計算の答えが、型で表せる範囲を超えること。debug は panic、release は既定で折り返す
`checked_add` / `checked_mul`	あふれたら `None` を返す計算。あふれたことをプログラムで扱える
`overflow-checks`	`[profile.release]` に書くと、release でもあふれを確かめる設定
アセットの束	`topcoat asset bundle` が作る、CSS などのファイルと目録（`manifest.toml`）。実行ファイルの隣の `assets\` に置く
再現性	同じソース・同じ道具・同じ部品から、いつ誰が作っても同じ物ができること
SHA-256 の指紋	ファイルの中身から計算した長い値。1 バイトでも違えば、まったく別の値になる。`Get-FileHash` で表示できる
`/Brepro`	Windows のリンカーに、時刻の代わりに中身から計算した値を使わせる指定
`--remap-path-prefix`	実行ファイルに入れるフォルダの場所の文字を、別の文字に置き換える rustc の指定
`RUSTFLAGS`	cargo が rustc に渡す追加の指定を書く環境変数。変えると全部のクレートが作り直しになる
os error 10048	Windows の「そのアドレスとポートは、もう使われている」エラー